



Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Compiladores - Prova 2 (Max: 20 pts)
Professor: Pedro Ramos

Estudante: _____

1. Tradução e geração de código (5pts)

Observe as regras de tradução para código intermediário de uma gramática com suporte para constantes e operação de soma (ADD).

#1	$R_i \leftarrow \text{Add}(R_i, R_j)$	custo: 2	código: ADDR Ri,Rj
#2	$R_i \leftarrow \text{Add}(R_i, K_c)$	custo: 3	código: ADDI Ri,c
#3	$R_i \leftarrow K_c$	custo: 2	código: LOADI Ri,c
#4	$R_i \leftarrow \text{Add}(R_i, \text{Add}(R_j, K_c))$	custo: 4	código: ADDRI Ri,Rj,c

Considere a expressão:

1 Add(Add(K1, K2), Add(K3, K4))

- Mostre a árvore de sintaxe abstrata desta expressão. (0.5pt)
 - Mostre a **sequência de regras de tradução** aplicadas na árvore da questão anterior que a transformam em código intermediário. (3pts)
 - Qual o **custo total** da tradução anterior (c) ? (0.5pt)
 - Explique por que um compilador atual nem sempre será capaz de gerar código ótimo para expressões como esta. (1pt)
2. **Alvo da compilação (2pts)** Linguagens de programação geralmente são compiladas para o código de máquina do computador de destino, mas às vezes um sistema interpretativo é usado. Discuta os méritos relativos a essas duas abordagens.
3. **Tipagem estática vs dinâmica (3pts)** Forneça um programa (use `let` para declarar variáveis) que execute sem erros de tipo em uma linguagem de tipagem dinâmica, mas é inválido em uma linguagem de tipagem estática como Java ou C++.

4. **Análise semântica (5pts)** Assinale cada afirmativa como correta (C) ou incorreta (I).

A análise semântica é responsável por converter o código fonte em uma árvore com informações de tipos nas expressões que, por sua vez, serão checadas em tempo de execução.	
Tipagem nominal permite polimorfismo, enquanto tipagem estrutural não permite polimorfismo.	
Herança é uma característica exclusiva de tipagem estrutural, uma vez que um layout de memória só pode ser expandido caso a estrutura do objeto seja conhecida em tempo de execução.	
Tipagem nominal checa os tipos durante a execução do programa, e tipagem estrutural checa os tipos em tempo de compilação.	
Máquina de pilha geralmente requer menos instruções para operações aritméticas, tornando-a mais eficiente que a máquina de registradores.	
Máquina de registradores tipicamente oferecem melhor performance devido ao acesso direto a registradores de propósito geral na CPU.	
A coerção de tipos durante a análise semântica ocorre da seguinte forma: o compilador converte todos os tipos para um tipo comum primitivo independentemente da sua posição no reticulado de herança.	
Durante a análise semântica, quando há incompatibilidade de operações entre dois tipos diferentes, o compilador visita recursivamente os nós na árvore abstrata do programa (AST) para identificar o menor ancestral comum (LUB) das expressões no reticulado de tipos, garantindo <i>soundness</i> e segurança.	
Para evitar problemas de conversão de tipos genérica, o compilador ignora o reticulado de tipos e utiliza regras ad-hoc para determinar cada conversão isoladamente.	
Herança múltipla exige que cada classe tenha uma tabela de métodos separada para cada classe-pai que o compilador mescla em tempo de execução.	
Herança múltipla complexifica a tabela de despacho de métodos ao exigir um mecanismo para resolver ambiguidade na chamadas de métodos, geralmente usando a linearização (Python) ou o modelo de herança virtual (C++).	

5. **Gere código MIPS para o seguinte segmento de código em C00L (5pts):**

```
1 int b[10];
2
3 int g(int x, int m) {
4     return x*m;
5 }
6
7 int f(int y) {
8     return g(y, 5) + b[6] + b[7];
9 }
10
11 f(2);
```

Obs: b é uma variável global que vive no endereço 0x600000.